



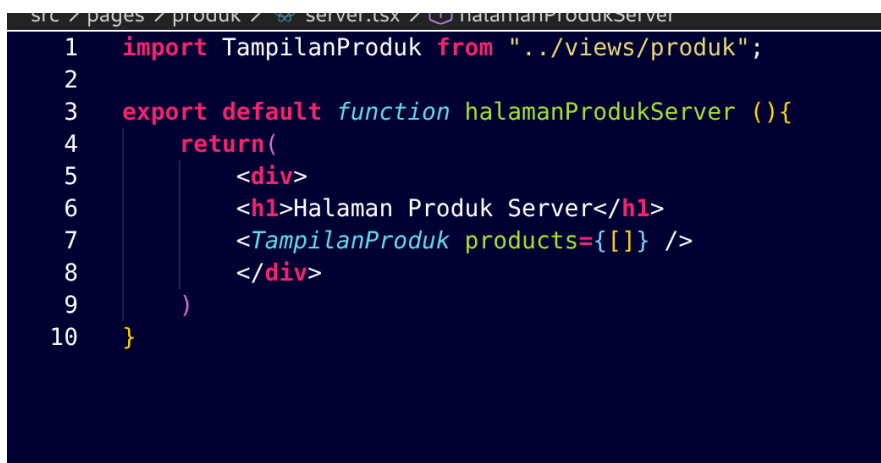
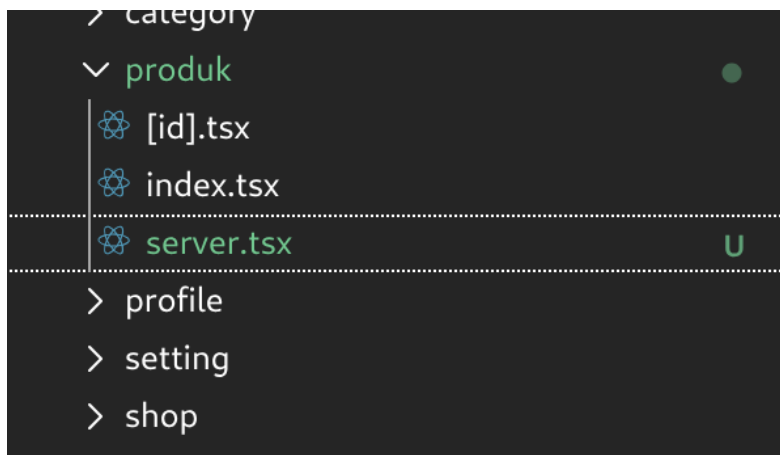
Mata Kuliah : Pemrograman Framework
Program Studi : D4 – Teknik Informatika
Semester : 6

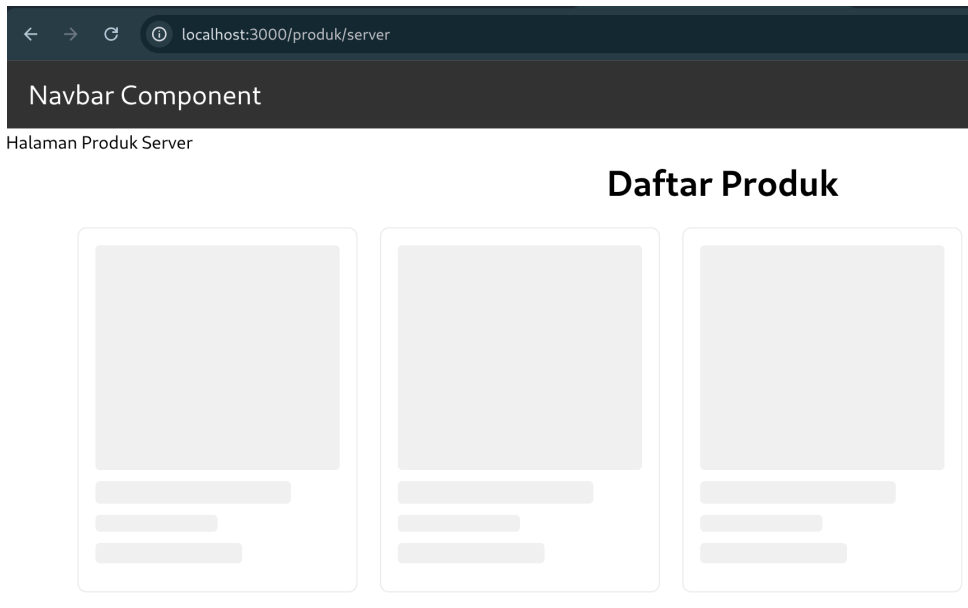
Kelas : TI-3C
NIM : 2341720165
Nama : Fandy Wahyu Hanzura
Jobsheet Ke- : 8

Laporan Jobsheet

Praktikum 1

1.





- Menambahkan client side server tsx pada halaman produk dengan mengambil layout pada halaman daftar produk

```
1 import TampilanProduk from "../views/produk";
2
3 type ProductType = {
4   id: string;
5   name: string;
6   price: number;
7   images: string;
8   category: string;
9 };
10
11
12 export default function halamanProdukServer (props: { products: ProductType[] }) {
13   const { products } = props;
14   return(
15     <div>
16       <h1>Halaman Produk Server</h1>
17       <TampilanProduk products={products} />
18     </div>
19   )
20 }
21
```

2.



Navbar Component

Halaman Produk Server

Daftar Produk



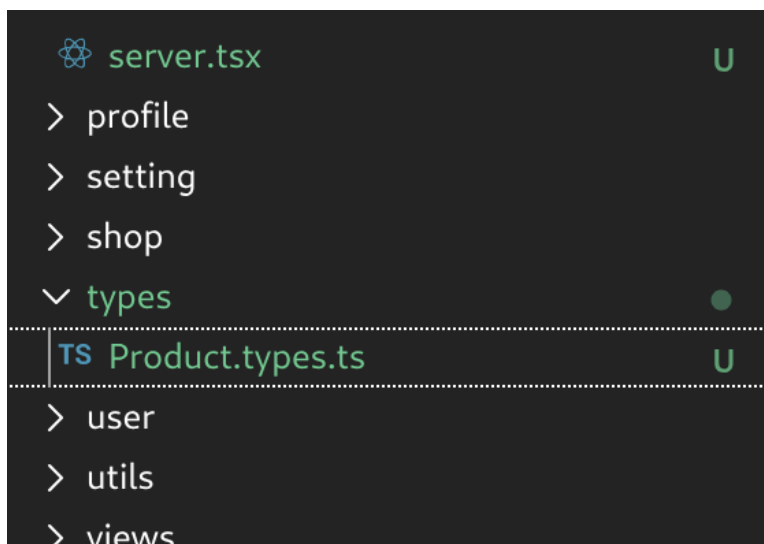
Baju Model
Baju Pria
Rp 150.000



Baju polo
Baju Anak
Rp 10.000

Yo its me Fandy Wahyu Hanzu

- Melakukan pemanggilan database yang ada pada halaman produk sehingga daftar bisa tampil



3.



```
1 export type ProductType = {
2   id: string;
3   name: string;
4   price: number;
5   images: string; // Sesuai data JSON API kamu
6   category: string;
7 };
```

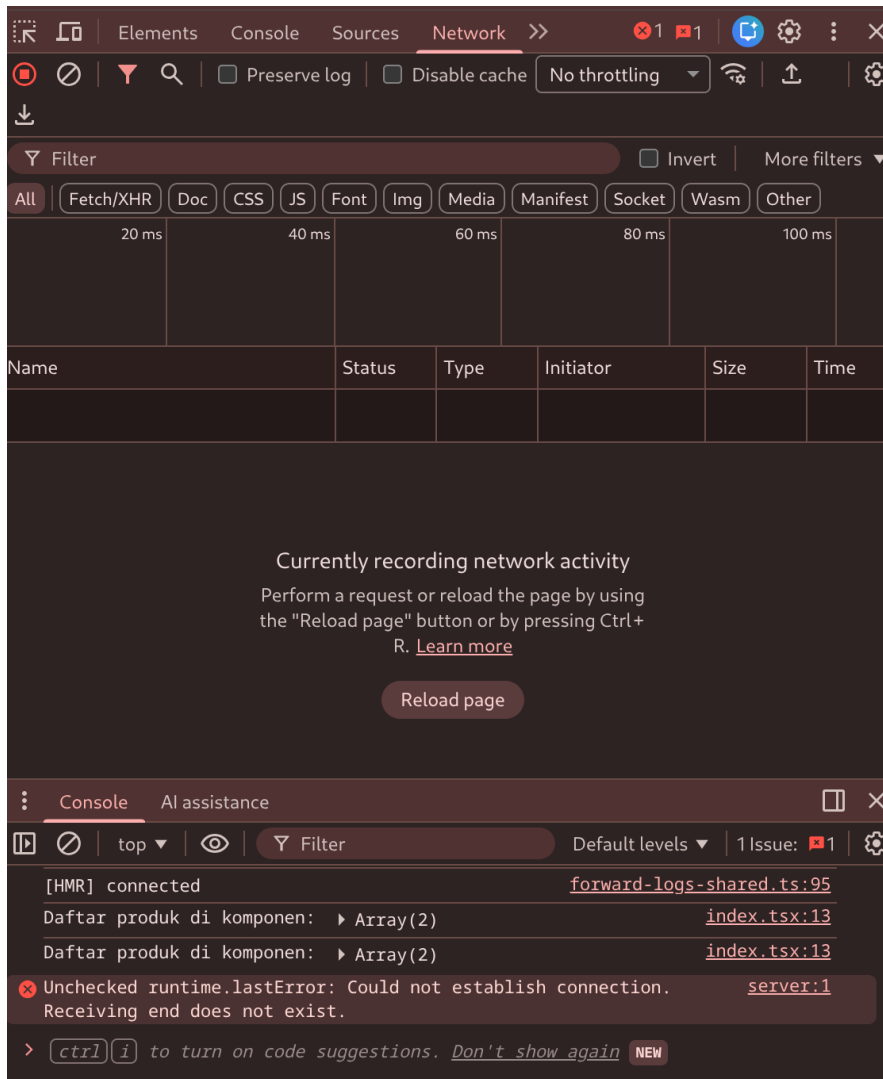
```
1 import TampilanProduk from "../views/produk";
2 import { ProductType } from "../types/Product.types";
3
4
5 export default function halamanProdukServer (props: { products: ProductType[] }) {
6   const { products } = props;
7   return(
8     <div>
9       <h1>Halaman Produk Server</h1>
10      <TampilanProduk products={products} />
11    </div>
12  )
13 }
14
15 export async function getServerSideProps() {
16   const res = await fetch('http://localhost:3000/api/produk');
17   const response = await res.json();
18
19   return {
20     props: {
21       products: response.data,
```

- Melakukan refactoring dengan membuat folder database sendiri yakni produktype.ts

```
react-dom-client.development.js:28004
Download the React DevTools for a better development experience:
https://react.dev/link/react-devtools

[HMR] connected forward-logs-shared.ts:95
Daftar produk di komponen: ▶ [] index.tsx:13
Daftar produk di komponen: ▶ [] index.tsx:13
Daftar produk di komponen: ▶ (2) [{...}, {...}] index.tsx:13
Daftar produk di komponen: ▶ (2) [{...}, {...}] index.tsx:13
```

4.



- ketika berada di halaman utama yakni produk akan muncul payload namun ketika di dalam halaman server side tidak akan muncul tanda tersebut

Praktikum mandiri

1. Berdasarkan seluruh rangkaian praktikum yang telah dilaksanakan, dapat disimpulkan bahwa pemilihan strategi rendering dalam framework Next.js—baik itu *Client-Side Rendering* (CSR) maupun *Server-Side Rendering* (SSR)—memiliki implikasi yang signifikan terhadap performa aplikasi, struktur kode, dan pengalaman pengguna (*User Experience*). Melalui observasi langsung pada *Developer Tools*, ditemukan bahwa CSR yang diimplementasikan menggunakan *hook* `useEffect` maupun library **SWR** bekerja dengan cara memuat dokumen HTML kosong terlebih dahulu ke browser. Fenomena ini tervalidasi



pada tab **Console**, di mana status awal daftar produk tercatat sebagai array kosong [] sebelum akhirnya terisi data setelah proses *fetching* selesai. Jeda waktu (latensi) yang muncul selama proses pengambilan data dari API, yang dalam pengujian tercatat sekitar **112ms**, mengharuskan adanya mekanisme *feedback* visual berupa **Skeleton Loading**. Penggunaan skeleton dengan animasi berkedip (*shimmer effect*) yang diatur melalui @keyframes identifier pada file SCSS terbukti efektif dalam mengurangi persepsi waktu tunggu pengguna dan menghindari tampilan layar kosong yang tidak informatif.

Lebih lanjut, implementasi library **SWR** memberikan lompatan optimasi yang besar dibandingkan pengambilan data manual menggunakan useEffect. SWR menerapkan strategi *Stale-While-Revalidate*, di mana aplikasi akan menampilkan data lama dari *cache* secara instan sambil melakukan validasi ulang di latar belakang. Hal ini terlihat dari perilaku tombol "Refresh Data" yang terasa statis dan halus (*no glitch*) karena SWR tidak menghapus data lama dari layar sebelum data baru diterima. Namun, jika diinginkan adanya *feedback* visual yang lebih tegas, penggunaan fungsi `mutate(undefined)` dapat memaksa aplikasi untuk kembali ke status *loading* dan menampilkan ulang skeleton. Selain itu, pengorganisasian kode dengan memisahkan fungsi fetcher ke dalam folder `utils/swr/fetcher.ts` mencerminkan praktik pengembangan profesional yang mengedepankan prinsip *reusability* dan kemudahan pemeliharaan kode (*maintainability*).

Perbandingan mendalam dilakukan dengan mengamati halaman **produk/server** yang menggunakan strategi **Server-Side Rendering (SSR)**. Perbedaan teknisnya sangat kontras jika dibandingkan dengan halaman CSR biasa. Pada halaman SSR, tab **Console** menunjukkan bahwa data produk langsung tersedia secara lengkap dalam bentuk array objek tanpa melalui fase array kosong []. Hal ini dikarenakan seluruh proses pengambilan data dari database Firebase telah diselesaikan di sisi server sebelum dokumen dikirimkan ke klien. Dampaknya, tab **Network** tidak lagi menunjukkan adanya *request fetch* tambahan ke `/api/produk` untuk konten utama. Secara visual, ini menghilangkan kebutuhan akan skeleton loading di sisi browser karena pengguna langsung menerima halaman yang sudah "matang" dan berisi informasi produk seutuhnya maupun gambar secara utuh. Meskipun SSR memberikan keuntungan dalam hal SEO dan kecepatan muat awal, CSR tetap menjadi pilihan yang sangat kuat untuk aplikasi yang sangat interaktif dan dinamis, terutama dengan dukungan library seperti SWR yang mampu menangani *caching* dan *revalidation* dengan sangat efisien.

Sebagai rangkuman akhir dari praktikum ini, integrasi antara Next.js, Firebase, dan SWR berhasil menciptakan aplikasi web yang responsif dan modern. Kemampuan untuk melakukan *conditional rendering* berdasarkan status `isLoading` dari SWR memungkinkan pengembang untuk mengontrol alur informasi yang diterima oleh pengguna secara presisi. Perbedaan antara CSR dan SSR bukan sekadar masalah teknis di tab *networking*, melainkan



pilihan strategis dalam arsitektur perangkat lunak untuk menyeimbangkan beban kerja antara server dan klien. Dengan memahami karakteristik masing-masing, pengembang dapat mengoptimalkan aplikasi baik dari sisi performa teknis (melalui pengurangan *request* dan *caching*) maupun dari sisi kenyamanan visual pengguna (melalui implementasi *skeleton loading* dan animasi transisi yang halus). Praktikum ini menegaskan bahwa dalam ekosistem Next.js, fleksibilitas untuk memilih strategi rendering per-halaman adalah salah satu fitur paling kuat untuk membangun aplikasi web berskala besar yang tetap cepat dan andal.

Pertanyaan refleksi

1. Mengapa SSR lebih baik untuk SEO?

- **Indexing Cepat:** Bot mesin pencari (seperti Googlebot) bisa langsung membaca teks, judul, dan deskripsi produk tanpa harus menjalankan JavaScript terlebih dahulu.
- **Kelemahan CSR di SEO:** Pada CSR, bot seringkali hanya melihat HTML kosong (`<div id="__next"></div>`) saat pertama kali datang, sehingga ada risiko konten tidak terindeks dengan sempurna jika bot tidak menunggu proses *fetching* selesai.

2. Kapan sebaiknya menggunakan SSR?

- **Halaman Publik:** Seperti katalog produk, artikel berita, atau blog yang sangat bergantung pada traffic dari mesin pencari (SEO).
- **Konten Dinamis yang Cepat Berubah:** Jika data harus selalu *up-to-date* setiap kali halaman dibuka tanpa jeda loading di sisi client.
- **Perangkat Low-End:** Karena beban render ada di server, perangkat pengguna yang speknya rendah tidak perlu bekerja keras menjalankan JS yang berat untuk menampilkan konten awal.

3. Apa kekurangan SSR dibanding CSR?

- **Beban Server Tinggi:** Server harus bekerja ekstra untuk merakit HTML setiap kali ada permintaan (*request*). Jika user-nya jutaan, server bisa cepat panas atau *down*.
- **Latensi TTFB (Time to First Byte):** User mungkin melihat layar putih sedikit lebih lama di awal karena server butuh waktu untuk mengambil data dari database sebelum bisa mengirim file HTML.
- **Interaktivitas:** Setelah halaman muncul, browser tetap harus melakukan proses **Hydration** (menjalankan JS) agar tombol-tombolnya bisa diklik.



4. Mengapa skeleton tidak muncul pada SSR?

- **Data Sudah "Ready"**: Skeleton itu fungsinya sebagai pengganti konten yang sedang diambil (*fetching*) di browser.
- **Tanpa Jeda Loading di Client**: Karena pada SSR proses pengambilan data sudah selesai di sisi server sebelum file dikirim, maka saat browser menerima file, data produk sudah ada di dalamnya.
- **Tidak Ada State Kosong**: Browser langsung merender produk secara utuh tanpa melewati fase array kosong `[]`, sehingga kondisi untuk memicu skeleton (`isLoading` atau `products.length === 0`) tidak pernah terpenuhi di sisi client.